Mejor reina que más obreros: costo y calidad de *swarms* de agentes LLM pequeños con un orquestador agnóstico al modelo y al dominio

Julián Andrés Mendoza Castro
Código A00395237
Universidad Icesi
Cali, Colombia
1109185879@u.icesi.edu.co

Esteban Gaviria Zambrano
Código A00396019
Universidad Icesi
Cali, Colombia
esteban.gaviria@u.icesi.edu.co

***Resumen*—Un *swarm* de agentes LLM reparte una tarea entre varios agentes que se comunican, y cada agente y cada mensaje consumen cómputo. Este trabajo presenta un orquestador reina–obreros que cambia de modelo y de dominio solo con configuración y lo usa para responder cuándo coordinar compensa su costo. Para que un *swarm* tuviera margen de mejorar, se usaron modelos pequeños que se equivocan con frecuencia: dos pares fuerte/débil (Qwen 2.5 de 3B/1.5B y Llama 3.2 de 3B/1B), ejecutados al 100 % en una misma GPU de 4 GB. Se compararon *swarms* jerárquicos y en malla con 2 y 4 obreros, con reina débil o fuerte, frente al agente único fuerte, al débil y a la votación por mayoría con el mismo número de llamadas, en GSM8K-Platinum, MATH-500 nivel 5 y un plan de viaje a Cartagena. En 3 640 corridas con hipótesis registradas de antemano, los *swarms* costaron entre 1.8× y 20.7× los GPU-segundos del agente débil, y la energía siguió a los GPU-segundos. Los *swarms* formados solo por el modelo débil no mejoraron al agente débil y quedaron por debajo de la votación en GSM8K (de -0.08 a -0.19). Con reina fuerte quedaron por debajo del modelo fuerte solo (de -0.07 a -0.23), a un costo de 2.4× a 12.0×. Cambiar solo la reina por el modelo fuerte sí mejoró la exactitud en GSM8K (de $+0.11$ a $+0.19$) y redujo los planes inválidos de Llama del 81–92 % al 22–39 % de las corridas. Con modelos pequeños, el cuello de botella es la coordinación: conviene un mejor coordinador o un modelo más capaz, no más agentes.**

***Palabras clave*—Sistemas multiagente, modelos de lenguaje, orquestación, costo–calidad, votación por mayoría, modelos locales, energía.**

I. Introducción

Los sistemas multiagente basados en LLM asignan roles a varios agentes que conversan para resolver una tarea [1]. AutoGen los modela como conversaciones programables [2], GPTSwarm como grafos optimizables [3], y herramientas como Ruflo los ofrecen como *swarms* con una reina y obreros [4]. Coordinar tiene un costo directo: Anthropic reporta que su sistema de investigación multiagente usa unas 15 veces los tokens de un chat [5]. También tiene un costo de fiabilidad: la taxonomía MAST atribuye la mayoría de los fallos de estos sistemas al diseño y a la coordinación [6]. Además, cuando el agente único ya resuelve más del 45 % de una tarea, agregar agentes da retornos negativos [7].

La pregunta de este trabajo es la que plantea la retroalimentación del curso: *¿agregar agentes y comunicación mejora la calidad lo suficiente para justificar el aumento de costo?* Un primer estudio nuestro con un modelo de API (`deepseek-flash`) no pudo responderla. El agente único resolvió el 100 % de las tareas, así que solo podía mostrarse que coordinar cuesta más [24]. Este trabajo cambia el diseño para que un *swarm* tenga margen de mejorar. Las contribuciones son cuatro:

- Un orquestador reina–obreros en Python, agnóstico al modelo y al dominio, publicado con todos sus registros [24].
- Un diseño de pares de capacidad con modelos locales, 100 % en GPU, que mide el cómputo (GPU-segundos) y la energía de cada llamada.
- Una comparación contra la línea base que suele faltar: la votación por mayoría del modelo débil con el mismo número de llamadas [10], [11]. La comparación usa hipótesis registradas antes de ejecutar y pruebas pareadas con corrección de Holm [15].
- Evidencia de que, con modelos pequeños, la ganancia viene de la reina y no del número de obreros.

II. Trabajo relacionado

La Tabla I ubica este trabajo. AutoGen reportó que su conversación multiagente supera a GPT-4 solo en MATH [2]. AgentPrune muestra que muchas aristas de comunicación sobran [8], y MacNet escala la colaboración a más de mil agentes [9]. Para un solo modelo, muestrear varias respuestas y votar mejora con el número de muestras [10], [11]. Por el teorema del jurado de Condorcet [12], la votación funciona mejor cuanto más independientes son los errores, pero los errores de los LLM están correlacionados [13]. Kim et al. [7] miden además que los sistemas multiagente amplifican los errores frente al agente único. Kapoor et al. [14] piden comparar agentes siempre contra su costo y contra líneas base simples.

Paper, curso Inteligencia Artificial (G1), Universidad Icesi, Cali, Colombia. Versión 2.0, 12 de octubre de 2026.

Tabla I
TRABAJOS RELACIONADOS Y LO QUE MIDE CADA UNO

Trabajo	Organización	Qué mide
AutoGen [2]	conversación entre agentes	exactitud (MATH, otras)
GPTSwarm [3]	grafo optimizable	exactitud y costo de optimizar
AgentPrune [8]	poda de aristas	tokens frente a exactitud
Votación [10], [11]	muestras independientes	exactitud frente a número de muestras
MAST [6]	7 marcos existentes	14 modos de fallo
Kim et al. [7]	5 arquitecturas	escalamiento y amplificación de errores
Este trabajo	único, jerárquica, malla; reina débil o fuerte	exactitud, GPU-segundos y energía frente a votación con igual número de llamadas

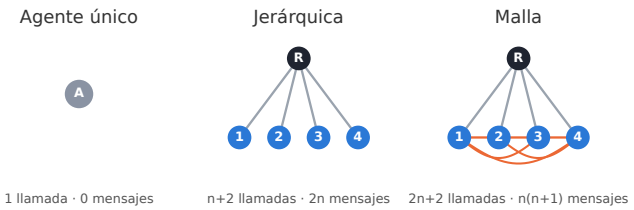


Figura 1. Topologías con $n = 4$ obreros. R es la reina; en la malla, las aristas naranjas forman la ronda de revisión entre obreros.

Este trabajo sigue esa recomendación con topologías fijas y cómputo medido en hardware.

III. METODOLOGÍA

III-A. Orquestador y topologías

Un agente es un rol, unas instrucciones y un acceso a un LLM a través de una interfaz única, `LLM.complete()`, que oculta el proveedor. Las topologías (Fig. 1) comparten cuatro etapas:

1. **Plan.** La reina reparte la tarea en una lista JSON con una subtarea por obrero. Si el JSON es inválido o no asigna una subtarea a cada obrero, se registra el fallo y cada obrero recibe la tarea completa.
2. **Trabajo.** Cada obrero resuelve su subtarea.
3. **Revisión** (solo en la malla). Cada obrero lee los borradores de todos y revisa el suyo.
4. **Integración.** La reina combina las contribuciones en la respuesta final.

El agente único responde directamente con el mismo formato final. En los dominios matemáticos, los obreros cierran con una línea `#### <número>`. Así se sabe si algún obrero acertó y si la reina conservó la respuesta correcta.

III-B. Diseño: pares de capacidad y líneas base

Cada familia aporta un modelo fuerte y uno débil: Qwen 2.5 de 3B y 1.5B [16] y Llama 3.2 de 3B y 1B [17], servidos con Ollama [18]. Un *swarm débil* usa el modelo débil en todos los agentes. Un *swarm mixto* usa el fuerte solo en la reina. Ambos se ejecutan en topología jerárquica y en malla con $n \in \{2, 4\}$ obreros. Las líneas base son el agente único fuerte, el agente único débil y la *votación*@k: la respuesta más frecuente entre k muestras independientes del modelo débil. La votación se empareja con cada *swarm* por número de llamadas. La jerárquica con $n = 2$ hace 4 llamadas y se compara con votación@3; la jerárquica con $n = 4$ y la malla con $n = 2$ hacen 6 y se comparan con votación@5; la malla con $n = 4$ hace 10 y se compara con votación@9. Con n obreros, la jerárquica hace $n + 2$ llamadas e intercambia $2n$ mensajes, y la malla hace $2n + 2$ llamadas y $n(n + 1)$ mensajes. Si cada borrador mide w tokens y el resto del *prompt* b, la revisión de la malla lee

$$t_{\text{rev}}^{\text{in}} \approx n\,b + n^2 w \tag{1}$$

tokens, así que su costo crece de forma cuadrática con n.

III-C. Hipótesis registradas

Antes de la primera corrida se fijaron ocho hipótesis y su prueba:

- **H1:** el costo crece con las llamadas y los mensajes.
- **H2:** el *swarm* débil cambia la exactitud del agente débil, en cualquier dirección.
- **H3:** un *swarm* supera a la votación con igual número de llamadas.
- **H4:** el *swarm* mixto alcanza al modelo fuerte solo.
- **H5:** la reina fuerte reduce los fallos de coordinación.
- **H6:** se describe cómo la reina conserva o pierde respuestas correctas.
- **H7:** los errores del modelo débil están correlacionados.
- **H8:** ambas familias ordenan igual las configuraciones.

También se fijó una regla de calibración. Un dominio es informativo para una familia si el agente débil resuelve entre el 10 % y el 90 % de 20 problemas de calibración, distintos de los de evaluación.

III-D. Dominios

La Tabla II resume los dominios. GSM8K-Platinum es la partición de prueba de GSM8K [19] con las etiquetas erróneas corregidas [20]. MATH-500 [21] es un subconjunto de MATH [22]; se usaron problemas de nivel 5 con respuesta entera. La tarea abierta es el caso de Cartagena: un viaje de 3 días para un estudiante con 600 000 COP. La calibración dejó GSM8K como informativo en ambas familias. MATH queda en nivel 5 porque la regla solo lo bajaba si ambas familias quedaban por debajo del 10 %, y solo Llama quedó en el piso (0 %). Por limitación de tiempo de cómputo, y antes de analizar resultados, MATH se redujo a 20 problemas con una repetición por *swarm* (enmienda B).

III-E. Protocolo y medición

Cada modelo se verificó al 100 % en la VRAM de una RTX 3050 Laptop de 4 GB, sin descarga a CPU, con ventana de 8 192 tokens. Para que el modelo de 3B de Llama cupiera hizo falta *flash attention* y una caché KV de 8 bits; sin ellas corría al 90 % en GPU. Se cargaba un solo modelo a la vez, se atendía una sola llamada a la vez, y el portátil estuvo conectado a la corriente en modo de rendimiento todo el experimento.

Tabla II
DOMINIOS Y CALIBRACIÓN (EXACTITUD DEL AGENTE DÉBIL EN 20 PROBLEMAS DE CALIBRACIÓN)

Dominio	Evaluación	Calidad	Qwen	Llama
GSM8K-Platinum	50 problemas; 2 rep. por *swarm*	coincidencia exacta	0.55	0.55
MATH-500 niv. 5	20 problemas; 1 rep. por *swarm*	coincidencia exacta	0.15	0.00
Cartagena	1 plan; 2 rep. por configuración	rúbrica de 7 criterios	–	–

Todas las configuraciones comparten temperatura 0.7, 1 024 tokens máximos de salida y los mismos *prompts*. El agente fuerte tiene 3 muestras por problema y el débil 9, de las que salen las votaciones. El cómputo de una corrida se mide en **GPU-segundos**: el tiempo de procesamiento del *prompt* más el de generación que reporta Ollama. La **energía** es la integral de la potencia de la GPU muestreada a 20 Hz por NVML. La potencia en reposo fue de 4.3 W, y las cargas de modelo se registran aparte. Cada corrida tiene un identificador derivado de su especificación, se ejecuta en orden aleatorio con semilla fija y se puede reanudar.

III-F. Evaluación y estadística

En los dominios matemáticos, la calidad es la coincidencia exacta del número final. Cartagena se califica con una rúbrica de siete criterios binarios con reglas de decisión explícitas:

- itinerario por día;
- presupuesto con total;
- total dentro del límite;
- alojamiento económico;
- dos riesgos con su mitigación;
- consistencia interna;
- lugares concretos.

El plan registrado preveía dos evaluadores humanos. Por decisión de los autores, las 40 respuestas, exportadas sin modelo ni configuración y en orden aleatorio, las calificó a ciegas un evaluador LLM (Claude Opus 5.5) antes de abrir la clave (enmienda C). **No es una evaluación humana** y tiene las limitaciones conocidas de los LLM como jueces [23]. Las diferencias entre configuraciones se calculan por problema, emparejadas, con intervalos *bootstrap* al 95 % y una prueba de permutación con cambio de signo. Las 80 comparaciones registradas se corrigen con Holm [15].

IV. RESULTADOS

Se ejecutaron 3 640 corridas de evaluación y 160 de calibración. Ninguna tuvo un fallo de infraestructura ni truncó el contexto; el *prompt* más largo fue de 4 281 tokens. La matriz completa tomó 10.0 horas de ejecución. La Tabla III reúne la exactitud y el cómputo de todas las configuraciones.

IV-A. Costo (H1)

Las llamadas y los mensajes siguen las fórmulas de la sección anterior. Frente al agente débil, los *swarms* usaron entre 1.8× y 20.7× los GPU-segundos en GSM8K y MATH, y hasta 26× los tokens. En Cartagena usaron entre 3.4× y 15.3× los GPU-segundos. La malla con $n = 4$ fue siempre la configuración más cara de su familia y equipo (Fig. 2). La energía siguió a los GPU-segundos, con 55 a 73 J por GPU-segundo en todas las configuraciones, así que el orden de costos es el mismo en ambas medidas. El costo es estructural: ambas familias ordenaron las configuraciones por costo casi igual (Kendall τ de 0.82 a 0.96, $p < 0.001$).

IV-B. Calidad frente a cómputo (H2–H4)

La Fig. 3 pone cada configuración en el plano exactitud–cómputo, y la Tabla IV resume las comparaciones registradas.

- **H2.** Ningún *swarm* débil mejoró al agente débil en GSM8K, con diferencias de -0.11 a -0.03. En MATH con Qwen, tres de las cuatro formas sumaron $+0.11$, pero los intervalos incluyen el cero.
- **H3.** Frente a la votación con el mismo número de llamadas, los *swarms* débiles quedaron por debajo en GSM8K: de -0.08 a -0.15 con Qwen y de -0.10 a -0.19 con Llama. Seis de las ocho diferencias tienen $p < 0.05$ sin corregir, pero ninguna sobrevive a Holm. Los *swarms* mixtos empataron con la votación.
- **H4.** El modelo fuerte solo fue mejor que cualquier *swarm* mixto: de 0.07 a 0.12 puntos más con Qwen y de 0.20 a 0.23 con Llama, mientras los mixtos usaban entre 2.4× y 5.9× (Qwen) y entre 6.4× y 12.0× (Llama) sus GPU-segundos. Son las únicas diferencias significativas tras Holm: 3 de 80, todas de Llama en GSM8K, con p_{Holm} entre 0.008 y 0.031.

En 4 de las 6 combinaciones de familia y dominio, la mejor configuración fue el agente único fuerte, con 1.5× a 1.7× el cómputo del débil. Las otras dos son Llama en MATH, en el piso (máximo 0.15), y Qwen en Cartagena (0.57 con dos respuestas por configuración); en ambas, un *swarm* queda primero dentro del ruido.

IV-C. Coordinación (H5, H6)

El único cambio que mejoró la exactitud fue la reina. Con la misma forma y los mismos obreros débiles, la reina fuerte subió la exactitud en GSM8K de $+0.12$ a $+0.17$ con Qwen y de $+0.11$ a $+0.19$ con Llama (seis de ocho con $p \leq 0.06$ sin corregir; ninguna tras Holm). El mecanismo difiere por familia (Tabla V, Fig. 4):

- **Llama.** La reina de 1B produjo un plan inválido en el 81–92 % de las corridas, de modo que su *swarm* débil mide sobre todo el modo de respaldo, en que cada obrero resuelve la tarea completa. Con reina de 3B, los planes inválidos bajaron al 22–39 %. Es el modo FM-1.1 de MAST, desobedecer la especificación [6].
- **Qwen.** Los planes ya eran válidos, y la ganancia está en la integración: la reina fuerte conservó la respuesta correcta de un obrero en el 67–73 % de las corridas, frente al 53–58 % de la débil.

En GSM8K, la reina descartó la respuesta correcta de un obrero en el 3–29 % de las corridas. La revisión de la malla aumentó

Tabla III

EXACTITUD (EX.) Y GPU-SEGUNDOS POR CORRIDA (GPU-S) POR FAMILIA Y DOMINIO, Y PUNTAJE DE LA RÚBRICA EN CARTAGENA (EVALUADOR LLM A CIEGAS). LA VOTACIÓN@k USA k MUESTRAS DEL MODELO DÉBIL. "JER" ES JERÁRQUICA; "DÉBIL" Y "MIXTO" INDICAN LA REINA DÉBIL O FUERTE, CON OBREROS SIEMPRE DÉBILES.

	Qwen, GSM8K		Llama, GSM8K		Qwen, MATH		Llama, MATH		Cartagena (rúbrica)	
Configuración	Ex.	GPU-s	Ex.	GPU-s	Ex.	GPU-s	Ex.	GPU-s	Qwen	Llama
único fuerte	0.83	1.7	0.74	1.9	0.33	7.7	0.10	6.7	0.36	0.86
único débil	0.62	1.1	0.46	1.1	0.14	4.5	0.07	4.9	0.50	0.71
votación@3 débil	0.66	3.2	0.52	3.3	0.15	13.8	0.10	14.8	–	–
votación@5 débil	0.74	5.6	0.54	5.5	0.10	22.5	0.05	24.7	–	–
votación@9 débil	0.72	9.8	0.58	10.0	0.25	40.2	0.05	44.4	–	–
jer-2 débil	0.58	3.1	0.42	6.5	0.25	8.0	0.10	16.8	0.36	0.64
jer-4 débil	0.59	6.1	0.36	10.1	0.10	16.0	0.10	23.1	0.36	0.36
malla-2 débil	0.59	4.1	0.35	9.1	0.25	11.9	0.15	20.9	0.57	0.36
malla-4 débil	0.59	8.8	0.39	16.4	0.25	22.0	0.00	41.0	0.29	0.43
jer-2 mixto	0.72	4.2	0.53	12.2	0.20	10.5	0.05	26.3	0.29	0.36
jer-4 mixto	0.73	7.9	0.53	15.9	0.25	18.9	0.05	31.6	0.21	0.57
malla-2 mixto	0.71	5.6	0.54	14.0	0.05	14.8	0.15	34.9	0.43	0.43
malla-4 mixto	0.76	10.3	0.51	22.9	0.15	28.9	0.00	44.8	0.57	0.79

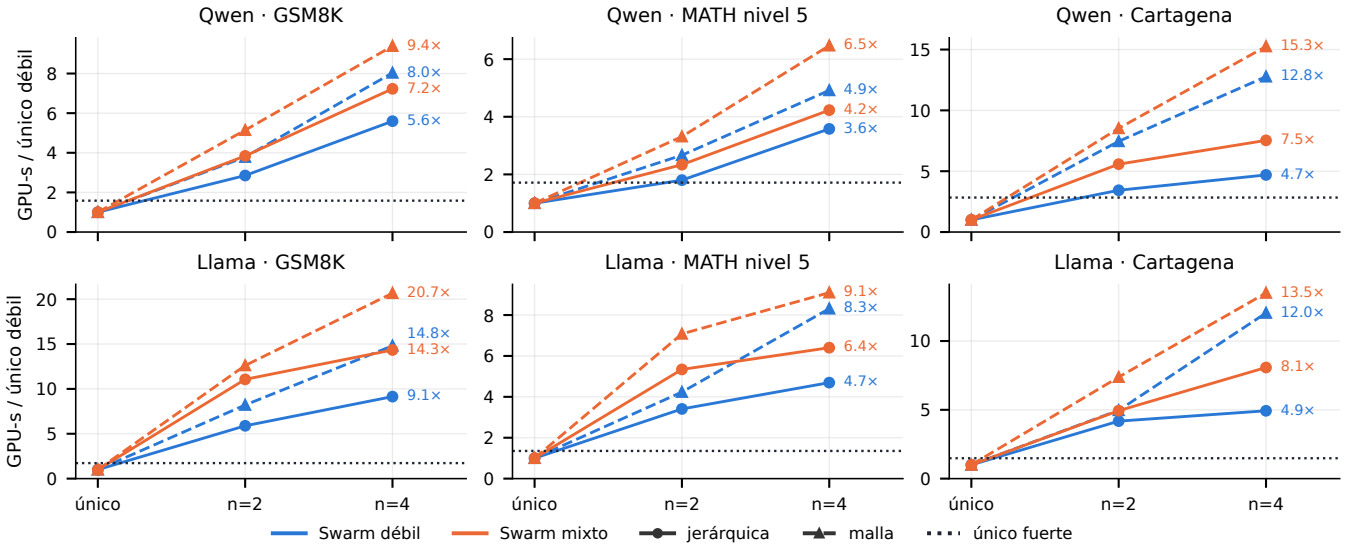


Figura 2. GPU-segundos por corrida relativos al agente único débil al pasar de $n = 2$ a $n = 4$ obreros. La línea punteada es el agente único fuerte. La malla (línea discontinua) crece más rápido que la jerárquica, y la reina fuerte añade costo.

el número de obreros correctos en el 20–43 % de las corridas y lo redujo en el 8–21 %. La mejora interna existe, pero la integración no siempre la aprovecha.

IV-D. Votación y correlación de errores (H7) y agnosticidad (H8)

Las muestras del modelo débil cometen errores correlacionados: la correlación media entre pares de muestras fue de $\phi = 0.24$ a 0.45. Por eso la votación ayuda pero se estanca. En Qwen y GSM8K pasó de 0.62 a 0.74 con $k = 5$ y bajó a 0.72 con $k = 9$, por debajo del 0.77 que predice Condorcet con errores independientes. Cuando la exactitud individual es menor que 0.5, el modelo binario de Condorcet no sirve de referencia, porque las respuestas erróneas se reparten entre muchos números y la mayoría relativa aún acierta. En cuanto a la agnosticidad, ambas familias ordenaron igual las configuraciones por costo. Por calidad coincidieron en GSM8K ($\tau = 0.58$, $p = 0.023$), pero no en MATH ($\tau = -0.13$) ni en Cartagena ($\tau = 0.15$).

IV-E. Cartagena

Con la rúbrica, el agente único obtuvo 0.61 de media, la malla 0.48 y la jerárquica 0.39. Trece de las 40 respuestas llegaron al límite de 1 024 tokens, casi todas de *swarms*, y obtuvieron 0.40 frente a 0.51 de las completas. El límite era igual para todos, y truncar es otro costo de integrar más texto. Casi ninguna respuesta propuso dos riesgos con mitigación (1 de 40) o un presupuesto consistente (4 de 40), y los modelos de 1–3B cometieron errores de hecho sobre la ciudad. Con dos

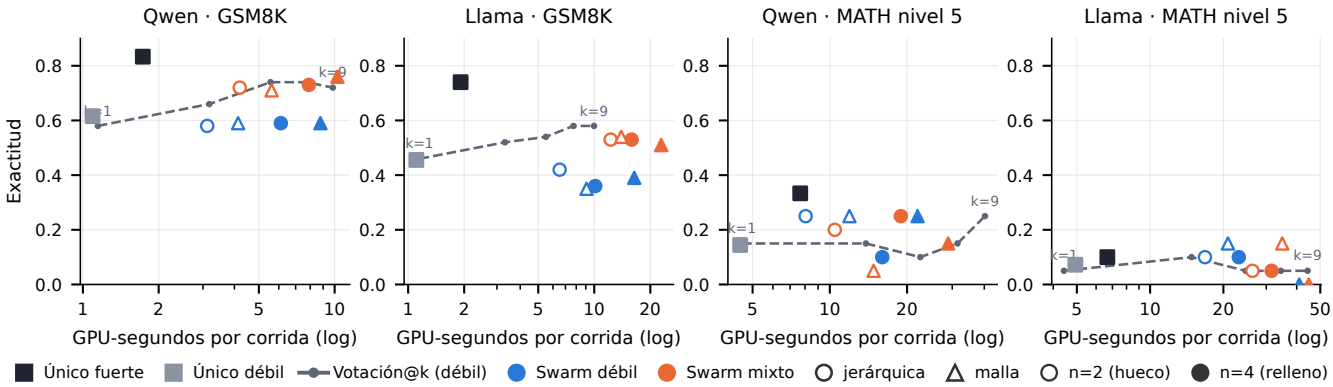


Figura 3. Exactitud frente a GPU-segundos por corrida (escala logarítmica). La línea discontinua es la votación del modelo débil con $k = 1, 3, 5, 7, 9$ muestras. Los *swarms* débiles (azul) quedan por debajo de esa curva y los mixtos (naranja) sobre ella, pero el agente único fuerte (cuadrado negro) los domina con menos cómputo.

Tabla IV
DIFERENCIAS DE EXACTITUD POR PROBLEMA (RANGO SOBRE LAS CUATRO FORMAS DE *swarm*), COMPARACIONES CON $p < 0.05$ SIN CORREGIR Y SIGNIFICATIVAS TRAS HOLM (DE 80 EN TOTAL)

Comparación	GSM8K		MATH		p<.05 Holm	
	Qwen	Llama	Qwen	Llama		
H2 swarm débil − único débil	−0.04 a −0.03	−0.11 a −0.04	−0.04 a +0.11	−0.07 a +0.08	1/16	0
H3 swarm débil − votación	−0.15 a −0.08	−0.19 a −0.10	+0.00 a +0.15	−0.05 a +0.10	6/16	0
H3 swarm mixto − votación	−0.03 a +0.06	−0.07 a +0.01	−0.10 a +0.15	−0.05 a +0.10	0/16	0
H4 swarm mixto − único fuerte	−0.12 a −0.07	−0.23 a −0.20	−0.28 a −0.08	−0.10 a +0.05	8/16	3
H5 swarm mixto − débil	+0.12 a +0.17	+0.11 a +0.19	−0.20 a +0.15	−0.05 a +0.00	5/16	0

Tabla V
COORDINACIÓN EN GSM8K: FRACCIÓN DE CORRIDAS (RANGO SOBRE LAS CUATRO FORMAS) Y MODO MAST MÁS CERCANO

Evento	Qwen		Llama		MAST
	débil	mixto	débil	mixto	
Plan inválido (JSON o roles)	0.02–0.12	0.06–0.20	0.81–0.92	0.22–0.39	FM-1.1 (parcial)
Reina descarta la respuesta correcta	0.05–0.17	0.03–0.09	0.16–0.29	0.03–0.18	FM-2.5/3.3 (cand.)
Reina conserva la respuesta correcta	0.53–0.58	0.67–0.73	0.25–0.36	0.36–0.42	–
Revisión de la malla mejora obreros	0.22–0.43	0.22–0.41	0.23–0.40	0.20–0.32	–
Revisión de la malla empeora obreros	0.11–0.19	0.14–0.21	0.08–0.14	0.12–0.13	–

respuestas por configuración y un solo evaluador LLM, este resultado es descriptivo.

V. DISCUSIÓN

La respuesta a la pregunta de investigación, con modelos de 1 a 3B, es negativa: agregar agentes y comunicación no compensó su costo. Hay dos alternativas más simples:

- Si hay un modelo más capaz disponible, usarlo solo fue mejor y entre 2.4× y 12.0× más barato que construir un *swarm* alrededor de él.

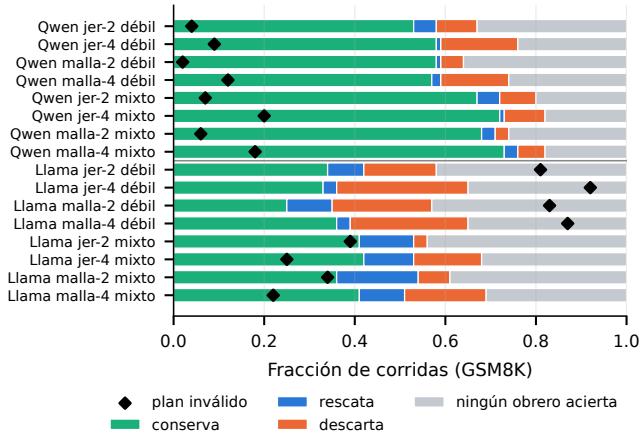


Figura 4. Qué hizo la reina con la respuesta de los obreros en GSM8K, por configuración. Los rombos marcan la fracción de planes inválidos.

- Si solo hay un modelo débil, votar entre k muestras fue al menos tan bueno como un *swarm* con el mismo número de llamadas.

El resultado es coherente con Kim et al. [7]: el agente débil resolvía el 46–62 % de GSM8K, por encima del umbral del 45 % donde agregar agentes deja de rendir.

Este estudio también muestra dónde está el problema. Los obreros acertaban con frecuencia, y la revisión de la malla los corregía. Lo que fallaba era la coordinación: planes que no seguían el formato y respuestas correctas descartadas al integrar. La única intervención que mejoró la exactitud fue darle a la reina un modelo más capaz. Para quien diseñe un *swarm* con modelos pequeños, la implicación práctica es invertir el cómputo en el coordinador o en un modelo mejor, no en más obreros. Además, todo *swarm* debería compararse contra la votación con el mismo número de llamadas.

Frente al primer estudio con `deepseek-flash`, la conclusión de que el agente único es la mejor opción se mantiene. Pero

ahora no se debe a un efecto techo: los *swarms* tenían margen para mejorar y no lo aprovecharon.

VI. Limitaciones y trabajo futuro

- **Alcance de los modelos.** Solo se probaron modelos de 1 a 3B en una GPU de 4 GB, con una temperatura y *prompts* no ajustados por modelo. Modelos mayores o *prompts* específicos podrían comportarse distinto.
- **Muestras pequeñas.** Las muestras son de 50 y 20 problemas, así que la mayoría de las comparaciones tiene poca potencia, y Holm sobre 80 comparaciones es conservador. Llama en MATH está en el piso y aporta poca información.
- **Modo de respaldo.** Cuando un plan es inválido, cada obrero recibe la tarea completa. Por eso el *swarm* débil de Llama se parece a muestrear varias veces e integrar.
- **Cartagena.** Es una sola tarea, con un evaluador LLM en lugar de dos humanos y respuestas truncadas en 1 024 tokens.
- **Energía.** Se midió solo la de la tarjeta gráfica.
- **Contaminación.** GSM8K y MATH pueden estar en los datos de entrenamiento.

Como trabajo futuro proponemos:

- reinas especializadas o con salida estructurada forzada;
- topologías podadas o adaptativas [3], [8];
- tareas que se puedan descomponer de verdad;
- evaluación humana con dos evaluadores.

VII. Conclusiones

Se construyó un orquestador reina–obreros agnóstico al modelo y al dominio, y se midió en GPU cuánto cuesta y cuánto aporta coordinar modelos pequeños. En 3 640 corridas, los *swarms* costaron entre $1.8\times$ y $20.7\times$ el cómputo del agente débil y no superaron ni a la votación con el mismo número de llamadas ni al modelo fuerte solo. Lo que sí mejoró la exactitud fue cambiar la reina. Con modelos pequeños, la calidad de un *swarm* depende de quién coordina más que de cuántos trabajan.

Disponibilidad de código y datos

El código, los cuadernos ejecutados (V4.0, con la matriz completa, y V4.2, con el análisis), los registros de cada corrida, llamada y carga de modelo, la evaluación ciega de Cartagena y el script que genera las tablas y figuras de este artículo están en un repositorio público [24]. El primer estudio con `deepseek-flash` queda archivado allí.

Referencias

[1] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," arXiv:2402.01680, 2024.

[2] Q. Wu, G. Bansal, J. Zhang *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. Conf. Language Modeling (COLM)*, 2024. arXiv:2308.08155.

[3] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "GPTSwarm: Language agents as optimizable graphs," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR 235, 2024.

[4] ruvnet, "Ruflo," repositorio de GitHub, 2026. https://github.com/ruvnet/ruflo

[5] Anthropic, "How we built our multi-agent research system," blog de ingeniería, 13 de junio de 2025. https://www.anthropic.com/engineering/multi-agent-research-system

[6] M. Cemri, M. Z. Pan, S. Yang *et al.*, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. arXiv:2503.13657.

[7] Y. Kim, K. Gu, C. Park *et al.*, "Towards a science of scaling agent systems," arXiv:2512.08296, 2025.

[8] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen, "Cut the crap: An economical communication pipeline for LLM-based multi-agent systems," arXiv:2410.02506, 2024.

[9] C. Qian, Z. Xie, Y. Wang *et al.*, "Scaling large language model-based multi-agent collaboration," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[10] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[11] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More agents is all you need," *Trans. Machine Learning Research (TMLR)*, 2024.

[12] M. J. A. N. de Caritat, marqués de Condorcet, *Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix*. París: Imprimerie Royale, 1785.

[13] E. Kim, A. Garg, K. Peng, and N. Garg, "Correlated errors in large language models," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025.

[14] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI agents that matter," arXiv:2407.01502, 2024.

[15] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[16] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[17] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[18] Ollama, "Ollama," software, versión 0.34.0, 2026. https://ollama.com

[19] K. Cobbe, V. Kosaraju, M. Bavarian *et al.*, "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.

[20] J. Vendrow, E. Vendrow, S. Beery, and A. Madry, "Do large language model benchmarks test reliability?" arXiv:2502.03461, 2025.

[21] H. Lightman, V. Kosaraju, Y. Burda *et al.*, "Let's verify step by step," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[22] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring mathematical problem solving with the MATH dataset," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.

[23] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[24] J. A. Mendoza Castro and E. Gaviria Zambrano, "swarm-llm-orchestrator," repositorio de GitHub, 2026. https://github.com/EstebanGZam/swarm-llm-orchestrator